

PROGRAMMING LANGUAGE USING JAVA 2

ARRAYS

Arrays are data structures in Java that store collections of elements of the same data type.

Array Types:

1. One-dimensional arrays (1D)
2. Multi-dimensional arrays (MD)
3. Jagged arrays (arrays with varying row lengths)

One Dimensional Array

Declaring Arrays:

1. `int[] arrayName;` (preferred)
2. `int arrayName[];`

Initializing Arrays:

1. `int[] arrayName = {1, 2, 3, 4, 5};`
2. `int[] arrayName = new int[5];`

Array Characteristics:

1. Fixed size
2. Homogeneous elements (same data type)
3. Indexed (access elements by index)
4. Length property (`arrayName.length`)

Array Operations:

1. Indexing (`arrayName[0]`)
2. Assignment (`arrayName[0] = 10`)
3. Length (`arrayName.length`)
4. Looping (for, while, foreach)
5. Searching (linear search, binary search)
6. Sorting (`Arrays.sort()`)

Array Methods:

1. `Arrays.toString()`: Converts array to string
2. `Arrays.sort()`: Sorts array in ascending order
3. `Arrays.binarySearch()`: Searches for element

4. `Arrays.copyOf()`: Copies array
5. `Arrays.fill()`: Fills array with value

Multi-Dimensional Arrays:

1. `int[][] arrayName = {{1, 2}, {3, 4}};`
2. `int[][] arrayName = new int[2][2];`

Jagged Arrays:

1. `int[][] arrayName = {{1, 2}, {3, 4, 5}};`

Example Code:

```
public class ArrayExample {  
    public static void main(String[] args) {  
        // Declare and initialize array  
        int[] scores = {90, 80, 70, 60};  
  
        // Access elements  
        System.out.println(scores[0]); // 90  
  
        // Modify element  
        scores[0] = 95;  
        System.out.println(scores[0]); // 95  
  
        // Loop through array  
        for (int score : scores) {  
            System.out.println(score);  
        }  
    }  
}
```

Types of an array formats

- (a) Array of primitive data types.

Array of Primitive Data Type:

In Java, an array of primitive data type stores multiple values of the same primitive type.

Primitive Data Types:

1. byte
2. short
3. int
4. long
5. float
6. double
7. boolean
8. char

Advantages of an array

1. Memory efficiency
2. Fast access and modification
3. Simple syntax

Disadvantages of an array

1. Fixed size
2. Limited functionality compared to object arrays

(b) Array of objects statement

Array of Objects Statement:

In Java, an array of objects stores multiple objects of the same class or interface.

Declaring Array of Objects:

1. ``ClassName[] objectArray;``
2. ``InterfaceName[] objectArray;``

Initializing Array of Objects:

1. ``ClassName[] objectArray = new ClassName[size];``
2. ``ClassName[] objectArray = {new ClassName(), new ClassName()};``

Example:

// Declare and initialize array of Student objects

```
Student[] students = new Student[3];
```

```
students[0] = new Student("John", 20);
```

```
students[1] = new Student("Jane", 22);
```

```
students[2] = new Student("Bob", 21);
```

```
// Alternatively
```

```
Student[] students = {new Student("John", 20), new Student("Jane", 22), new Student("Bob", 21)};
```

```
...
```

Characteristics:

1. Fixed size
2. Homogeneous elements (same class or interface)
3. Indexed (access elements by index)
4. Length property (`objectArray.length`)

Methods:

1. `objectArray[index].methodName()`
2. `objectArray[index].propertyName`

Advantages:

1. Stores multiple objects of same class/interface
2. Easy access and manipulation of objects
3. Memory efficient

Disadvantages:

1. Fixed size
2. Requires manual memory management

Best Practices:

1. Use meaningful variable names
2. Initialize arrays with meaningful objects
3. Use arrays instead of collections for fixed-size data
4. Avoid using arrays for large datasets

Common Operations:

1. Indexing
2. Assignment
3. Length
4. Looping
5. Searching
6. Sorting

Explain with illustration the storage and retrieval process of a 1-dimensional and 2-dimensional array

Here's an explanation of the storage and retrieval process for 1-dimensional and 2-dimensional arrays:

1-Dimensional Array

Storage Process:

1. Memory allocation: Contiguous blocks of memory are allocated for the array.

```
...  
  
+-----+-----+-----+-----+-----+  
| Index 0 | Index 1 | Index 2 | Index 3 | Index 4 |  
+-----+-----+-----+-----+-----+  
| Value   | Value   | Value   | Value   | Value   |  
+-----+-----+-----+-----+-----+  
...
```

2. Values are stored in each index.

```
...  
  
+-----+-----+-----+-----+-----+  
| Index 0 | Index 1 | Index 2 | Index 3 | Index 4 |  
+-----+-----+-----+-----+-----+  
|   10    |   20    |   30    |   40    |   50    |  
+-----+-----+-----+-----+-----+  
...
```

Retrieval Process:

1. Provide the index of the desired value.

2. The value at the specified index is retrieved.

array[2] = 30

2-Dimensional Array

Storage Process:

1. Memory allocation: Contiguous blocks of memory are allocated for the array.

```
+-----+-----+-----+
| Row 0   | Row 0   | Row 0   |
+-----+-----+-----+
| Col 0   | Col 1   | Col 2   |
+-----+-----+-----+
| Value   | Value   | Value   |
+-----+-----+-----+
|         |         |         |
+-----+-----+-----+
| Row 1   | Row 1   | Row 1   |
+-----+-----+-----+
| Col 0   | Col 1   | Col 2   |
+-----+-----+-----+
| Value   | Value   | Value   |
+-----+-----+-----+
...
```

2. Values are stored in each row and column.

```
...

+-----+-----+-----+
| Row 0   | Row 0   | Row 0   |
+-----+-----+-----+
| Col 0   | Col 1   | Col 2   |
+-----+-----+-----+
| 10      | 20      | 30      |
```

Row 1	Row 1	Row 1	
Col 0	Col 1	Col 2	
40	50	60	

Retrieval Process:

1. Provide the row and column indices of the desired value.
2. The value at the specified row and column is retrieved.

`array[0][1] = 20`

Here is a simple code snippet illustrating storage and retrieval in 1D and 2D arrays:

```
public class ArrayExample {
    public static void main(String[] args) {
        // 1D Array
        int[] oneDArray = {10, 20, 30, 40, 50};
        System.out.println("1D Array Value: " + oneDArray[2]);

        // 2D Array
        int[][] twoDArray = {{10, 20, 30}, {40, 50, 60}};
        System.out.println("2D Array Value: " + twoDArray[0][1]);
    }
}
```

Explain how to create Stack and Queue data structure

Here's an explanation of how to create Stack and Queue data structures:

Stack Data Structure

A Stack is a Last-In-First-Out (LIFO) data structure, meaning the last element added to the stack will be the first one to be removed.

Creating a Stack

Here are the steps to create a Stack:

1. ***Choose a programming language***: You can implement a Stack in any programming language, such as Java, Python, or C++.
2. ***Define the Stack class***: Create a new class called `Stack` that will contain the Stack's elements and methods.
3. ***Define the Stack's elements***: The Stack will contain a collection of elements, such as an array or a linked list.
4. ***Define the Stack's methods***: The Stack will have methods to perform operations, such as:
 - `push(element)` : Adds an element to the top of the Stack.
 - `pop()` : Removes the top element from the Stack.
 - `peek()` : Returns the top element without removing it.
 - `isEmpty()` : Returns true if the Stack is empty, false otherwise.
5. ***Implement the Stack's methods***: Write the code to implement each of the Stack's methods.

Example Stack Implementation

Here's an example implementation of a Stack in Java:

```
import java.util.Stack;

public class Stack {

    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();

        stack.push(23);
        stack.push(24);
        stack.push(25);

        System.out.println(stack.pop());
        System.out.println(stack.pop());
    }
}
```

Output: 23

Output: 24

Queue Data Structure

A Queue is a First-In-First-Out (FIFO) data structure, meaning the first element added to the queue will be the first one to be removed.

Creating a Queue

Here are the steps to create a Queue:

1. ***Choose a programming language***: You can implement a Queue in any programming language, such as Java, Python, or C++.
2. ***Define the Queue class***: Create a new class called `Queue` that will contain the Queue's elements and methods.
3. ***Define the Queue's elements***: The Queue will contain a collection of elements, such as an array or a linked list.
4. ***Define the Queue's methods***: The Queue will have methods to perform operations, such as:
 - `enqueue(element)`: Adds an element to the end of the Queue.
 - `dequeue()`: Removes the front element from the Queue.
 - `peek()`: Returns the front element without removing it.
 - `isEmpty()`: Returns true if the Queue is empty, false otherwise.
5. ***Implement the Queue's methods***: Write the code to implement each of the Queue's methods.

Example Queue Implementation

Here's an example implementation of a Queue in Java:

```
import java.util.Queue;
import java.util.LinkedList;

public class Que{
    public static void main(String[] args){
        Queue<Integer> q= new LinkedList<>();
        q.add(1);
        q.add(2);
        q.add(3);
        System.out.println(q.peek());
        System.out.println(q.remove());
```

Output: 1

Output: 2

Develop a simple array structure program to

(a) manipulate objects using vectors, but before going into the program let's understand vectors in Java:

What is a vector in Java:

In Java, a Vector is a type of data structure that is similar to an array but no size limit and can store any number of elements and has an `add()` method. Here are the key characteristics of a Vector in Java:

Key Features of Vectors in Java

1. ***Dynamic size***: Vectors can grow or shrink dynamically as elements are added or removed.
2. ***Index-based access***: Elements in a Vector can be accessed using an index, just like in an array.
3. ***Thread-safe***: Vectors are thread-safe, meaning they can be safely accessed and modified by multiple threads concurrently.

Methods of Vector Class

Here are some commonly used methods of the Vector class:

1. ***addElement()***: Adds an element to the end of the Vector.
2. ***insertElementAt()***: Inserts an element at a specified index.
3. ***removeElement()***: Removes the first occurrence of an element.
4. ***removeElementAt()***: Removes the element at a specified index.
5. ***setSize()***: Sets the size of the Vector.
6. ***size()***: Returns the current size of the Vector.
7. ***elementAt()***: Returns the element at a specified index.
8. ***contains()***: Returns true if the Vector contains a specified element.

Example Usage

Here's an example of using a Vector:

...

```
import java.util.Vector;

public class VectorExample {
    public static void main(String[] args) {
        // Create a Vector
        Vector<String> vector = new Vector<>();
```

```

// Add elements to the Vector
vector.addElement("Apple");
vector.addElement("Banana");
vector.addElement("Cherry");

// Print the Vector
System.out.println("Vector: " + vector);

// Remove an element from the Vector
vector.removeElement("Banana");

// Print the updated Vector
System.out.println("Updated Vector: " + vector);
}
}
...

# Output
...

```

Vector: [Apple, Banana, Cherry]

Updated Vector: [Apple, Cherry]

In summary, Vectors in Java are dynamic, index-based, and thread-safe data structures that provide a range of methods for adding, removing, and accessing elements.

(a) Manipulate objects using vectors,

Here's a simple array structure program in Java that manipulates objects using vectors:

```
# Array Structure Program
```

```
import java.util.Vector;
```

```
// Define a class to represent objects
```

```

class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public String toString() {
        return "Name: " + name + ", Age: " + age;
    }
}

public class ArrayStructureProgram {
    public static void main(String[] args) {
        // Create a vector to store Person objects
        Vector<Person> personVector = new Vector<>();

        // Add Person objects to the vector
        personVector.add(new Person("John Doe", 30));
        personVector.add(new Person("Jane Doe", 25));
        personVector.add(new Person("Bob Smith", 40));

        // Print the vector
        System.out.println("Vector:");
        for (Person person : personVector) {
            System.out.println(person);
        }

        // Manipulate objects in the vector
        personVector.set(0, new Person("John Doe Updated", 31)); // Update the first element
        personVector.remove(1); // Remove the second element
        personVector.add(new Person("Alice Johnson", 35)); // Add a new element
    }
}

```

```

        // Print the updated vector
        System.out.println("\nUpdated Vector:");
        for (Person person : personVector) {
            System.out.println(person);
        }
    }
}

```

Output

Vector:

Name: John Doe, Age: 30

Name: Jane Doe, Age: 25

Name: Bob Smith, Age: 40

Updated Vector:

Name: John Doe Updated, Age: 31

Name: Bob Smith, Age: 40

Name: Alice Johnson, Age: 35

This program demonstrates the following operations:

- Creating a vector to store `Person` objects.
- Adding `Person` objects to the vector using the `add()` method.
- Printing the vector using a for-each loop.
- Updating an element in the vector using the `set()` method.
- Removing an element from the vector using the `remove()` method.
- Adding a new element to the vector using the `add()` method.

Define methods that accept array as pointers in simple JAVA program

An array pointer method is a technique used in programming to manipulate and access array elements using pointers. In this approach, an array is treated as a collection of memory locations, and a pointer is used to traverse and access these locations.

How Array Pointer Methods Work

Here's a step-by-step explanation of how array pointer methods work:

1. ***Array Declaration***: An array is declared with a specified size, and memory is allocated for each element.
2. ***Pointer Declaration***: A pointer variable is declared to point to the array.
3. ***Pointer Arithmetic***: The pointer is incremented or decremented using pointer arithmetic to traverse the array elements.
4. ***Array Element Access***: The pointer is used to access and manipulate individual array elements.

Benefits of Array Pointer Methods

Array pointer methods offer several benefits:

1. ***Efficient Memory Access***: Array pointer methods allow for efficient memory access, as the pointer can be incremented or decremented to access adjacent elements.
2. ***Flexible Array Manipulation***: Array pointer methods provide flexibility in manipulating array elements, as the pointer can be used to access and modify elements in a variety of ways.
3. ***Improved Performance***: Array pointer methods can improve performance in certain applications, as they eliminate the need for explicit array indexing.

Example of Array Pointer Method in Java to work with arrays.

Here's an example of how you can manipulate and access array elements in Java:

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {10, 20, 30, 40, 50};  
  
        // Print array elements using indexing  
        for (int i = 0; i < arr.length; i++) {  
            System.out.print(arr[i] + " ");  
        }  
        System.out.println();  
  
        // Modify array elements using indexing  
        arr[2] = 300; // Modify the third element  
  
        // Print modified array elements  
        for (int i = 0; i < arr.length; i++) {  
            System.out.print(arr[i] + " ");  
        }  
        System.out.println();  
    }  
}
```

```
    }  
}
```

Output

10 20 30 40 50

10 20 300 40 50

In this example, we declare an array `arr` and use indexing to access and modify its elements.

Java's Array Pointer Limitations

While Java provides some functionality similar to array pointer methods, it has some limitations:

1. ***No Explicit Pointer Arithmetic***: Java does not support explicit pointer arithmetic like C or C++.
2. ***Array Bounds Checking***: Java performs array bounds checking to prevent out-of-bounds access, which can lead to exceptions.
3. ***No Pointer Aliasing***: Java does not support pointer aliasing, which means that multiple pointers cannot point to the same memory location.